

Multi-Asset Market Dashboard

Build Notes: Process, Tools, and Implementation

This document describes how the second dashboard in this project was built: a single interactive HTML page that lets the user switch between five assets — Gold, Bitcoin, Ethereum, Apple, and Amazon — using simple tabs, with all data sourced from the `Power_BI_Data.xlsx` workbook. Unlike the earlier Gold-only dashboard, this version deliberately leaves out any forecasting panel, per the brief for this build.

1. Tools Used

The build relies on a small, standard set of tools: Python for all data preparation, and a single self-contained HTML page for the interface. Nothing here requires a server — once the page loads, all five assets' data already live inside it.

Tool / Library	Role in the Build
Python 3.12	All data cleaning, aggregation, and JSON export
pandas / openpyxl	Reading each sheet of the uploaded workbook, parsing dates and mixed-format volume text
JSON	Packaging KPIs and yearly / quarterly / monthly / weekly series for all five assets into one embedded data object
HTML5 / CSS3	Page layout, card styling, and theming via CSS custom properties
JavaScript (vanilla)	Asset tabs, year/metric filters, and rebuilding charts on switch — entirely client-side
Chart.js 4.4.0 (via CDN)	The quarterly bar chart, monthly area chart, full-history line chart, and volume bar chart
docx (npm)	Generating this write-up

2. What the Uploaded File Actually Contained

The uploaded workbook, `Power_BI_Data (1).xlsx`, turned out to contain five sheets rather than the Silver and Oil data originally requested: Gold, Bitcoin, Ethereum, Apple, and Amazon. Each sheet shares the same column layout — Date, Price, Open, High, Low, Vol., and Change % — which made a single shared cleaning and aggregation pipeline possible across all five.

Asset	Rows	Reported Date Range
Gold	2,081	2017-01-02 to 2026-12-06
Bitcoin	3,489	2017-01-01 to 2026-12-07
Ethereum	3,489	2017-01-01 to 2026-12-07
Apple	2,398	2017-01-02 to 2026-12-06

Asset	Rows	Reported Date Range
Amazon	2,398	2017-01-02 to 2026-12-06

The dashboard was built for the five assets actually present — covering both assets originally requested (Bitcoin, Apple) plus Ethereum, Amazon, and Gold as a bonus. Silver and Oil can be added later using the same pipeline, once separate data for those is available.

Data verification note (please check before final submission):

The date range above ends in December 2026 for all five assets, which is later than the current date and mirrors a day/month parsing issue identified earlier in this project on the Gold sheet (where, for example, '08/01' was being read as August 1st instead of January 8th). That earlier case was traced and corrected, and the verified range for Gold turned out to be 2017-01-02 to 2026-07-21. Since all five sheets here show the same end-of-year pattern, it's worth re-running that same date correction across all five sheets before relying on "latest_date," "ytd_change_pct," or any of the quarterly/monthly/weekly trend charts, since those specifically depend on the dates being in the right order. Row counts and high/low values are less likely to be affected, since those don't depend on date ordering.

3. Cleaning the Data

The 'Vol.' column mixed suffixed text values — things like '55.09K', '102.85K', or '53.47M' — rather than plain numbers, so each value needed to be converted before anything could be aggregated:

```
def parse_vol(v):
    if pd.isna(v): return 0.0
    v = str(v).strip()
    mult = 1
    if v.endswith('K'): mult = 1e3; v = v[:-1]
    elif v.endswith('M'): mult = 1e6; v = v[:-1]
    elif v.endswith('B'): mult = 1e9; v = v[:-1]
    try:
        return float(v) * mult
    except:
        return 0.0

df['Volume'] = df['Vol.'].apply(parse_vol)
df['Close'] = df['Price'].astype(float)
```

4. Aggregating Each Asset

The same approach used for the earlier Gold-only dashboard was reused here: for every sheet, four aggregations were computed — yearly, quarterly, and monthly summaries, plus a weekly resample to keep the long-run trend line smooth rather than noisy.

```
yearly = df.groupby('Year').agg(
    open=('Open', 'first'), close=('Close', 'last'),
    high=('High', 'max'), low=('Low', 'min'),
```

```

    avg=('Close','mean'), volume=('Volume','sum')
).reset_index()

quarterly = df.groupby(['Year','Quarter']).agg(
    close=('Close','last'), avg=('Close','mean'), volume=('Volume','sum')
).reset_index()
quarterly['label'] = quarterly['Year'].astype(str) + ' Q' +
quarterly['Quarter'].astype(str)

monthly = df.groupby('YM').agg(
    close=('Close','last'), avg=('Close','mean'), volume=('Volume','sum')
).reset_index()

weekly = df.set_index('Date').resample('W').agg(
    close=('Close','last'), volume=('Volume','sum')
).reset_index()

```

5. Per-Asset KPIs

Alongside the time-series aggregates, a small set of headline numbers is computed for each asset — all-time high and low, latest close, total and average volume, record count, and year-to-date change:

```

kpi = {
    'all_time_high': round(df['High'].max(), 2),
    'all_time_low': round(df['Low'].min(), 2),
    'latest_close': round(latest['Close'], 2),
    'latest_date': latest['Date'].strftime('%Y-%m-%d'),
    'total_volume': int(df['Volume'].sum()),
    'avg_close': round(df['Close'].mean(), 2),
    'records': len(df),
    'ytd_change_pct': round((latest['Close'] / year_start_close - 1) * 100, 2)
}

```

The resulting all-time high and low for each asset, taken from the High and Low columns:

Asset	All-Time High	All-Time Low
Gold	\$5,626.80	\$1,146.50
Bitcoin	\$126,186.00	\$739.50
Ethereum	\$4,955.90	\$8.03
Apple	\$334.99	\$28.69
Amazon	\$278.56	\$37.38

6. One Shared JSON Object for All Five Assets

Rather than exporting five separate files, every asset's aggregates were nested under its own key inside a single JSON object, keyed by sheet name, and embedded directly into the HTML page:

```

assets = {}
for sheet_name in ['Gold', 'Bitcoin', 'Ethereum', 'Apple', 'Amazon']:
    df = xls.parse(sheet_name)
    # ...clean + aggregate as above...
    assets[sheet_name] = {
        'symbol': symbol_map[sheet_name],
        'yearly': yearly.round(2).to_dict(orient='records'),
        'quarterly': quarterly.round(2).to_dict(orient='records'),
        'monthly': monthly.round(2).to_dict(orient='records'),
        'weekly': weekly.round(2).to_dict(orient='records'),
        'kpi': kpi
    }

data_json = json.dumps(assets)  # embedded directly into the HTML file

```

7. Building the Tabbed Dashboard

The page itself renders one shared set of KPI cards and charts, and a small row of tabs simply re-points those same charts at a different asset's slice of the JSON object — there's no page reload and no server involved:

```

const tabsEl = document.getElementById('assetTabs');
tabsEl.innerHTML = ASSET_KEYS.map(k =>
  `<div class="asset-tab" data-asset="${k}">${META[k].label}</div>`
).join('');

tabsEl.querySelectorAll('.asset-tab').forEach(tab => {
  tab.addEventListener('click', () => {
    currentAsset = tab.getAttribute('data-asset');
    renderAsset();  // repopulates KPIs and rebuilds all four charts
  });
});

```

Number formatting also adapts automatically per asset, since a fixed number of decimal places doesn't work well when Bitcoin's all-time high is roughly \$126,000 and Ethereum's all-time low is roughly \$8 — the same formatting function handles both sensibly:

```

const fmt = (n, unit) => unit + n.toLocaleString(undefined, {
  maximumFractionDigits: Math.abs(n) < 10 ? 2 : 0
});

```

8. What Changed from the Gold-Only Dashboard

- Added: asset tabs (Gold / Bitcoin / Ethereum / Apple / Amazon) at the top of the page.
- Added: per-asset number formatting, so both small-value and large-value assets display cleanly side by side.
- Removed: the 3-month forecast panel and its underlying ETS model — intentionally left out of this build, as requested.
- Kept: the same KPI cards, quarterly bar chart, monthly area chart, full-history trend line, and volume trend — now reused across all five assets instead of just one.

9. Known Gap

Silver and Oil were part of the original request but weren't present in the uploaded workbook. If historical data for either is provided — a CSV or Excel export from a source like Yahoo Finance would work fine — both can be added as two further tabs using the exact same pipeline described above.

10. Where Everything Lives

- `multi_asset_dashboard.html` — the interactive dashboard itself. It opens in any browser, and all data is already embedded in the file; only the `Chart.js` CDN script needs an internet connection.
- Source data: `Power_BI_Data (1).xlsx`, sheets Gold / Bitcoin / Ethereum / Apple / Amazon.